

Recipe Creation Agent

IN PROGRESS

June 2026 — Ongoing · AI · Full-Stack Backend · Python · FastAPI · PostgreSQL · Gemini

Anshumaan Sai Patnaik

Project status: nearing completion. The FastAPI backend is implemented across five development phases and covered by an extensive test suite; a Next.js manager dashboard is in place. Remaining work is final integration, polish, and deployment. This is the most ambitious project in the collection.

Overview

The Recipe Creation Agent is an AI-driven recommendation engine for restaurant kitchens. Given a kitchen's live inventory and real-world context — weather, local events, time of day — it decides which dishes are the most profitable and practical to cook right now, explains each choice in plain language, and can even draft brand-new recipes from whatever is in stock.

It is built as a multi-tenant HTTP API (23 endpoints) backed by PostgreSQL, with a learning loop that improves recommendations over time from real outcomes, and a separate manager dashboard for analytics. It combines a classic data/scoring pipeline with embeddings, demand forecasting, a learning-to-rank model, and large-language-model generation.

The Problem

Restaurant kitchens decide what to cook largely by habit and guesswork. The result is waste (ingredients expire unused), missed margin (cooking dishes that don't sell, or that are expensive to make), and no systematic response to context — a rainy evening, a cricket match nearby, or a festival all change what will actually sell, but a busy kitchen can't reason about all of that on the fly.

The agent turns that judgement into a data-driven decision. It scores every makeable dish against the kitchen's current stock and conditions, prioritises ingredients about to expire to cut waste, accounts for cost, demand, and the chef's strengths, and learns from what sold well last time — giving the kitchen a ranked, explained shortlist instead of a hunch.

Technology Stack

Python + FastAPI	Async HTTP API with Pydantic-typed request/response contracts and auto-generated OpenAPI docs.
PostgreSQL + SQLAlchemy	Relational store for restaurants, recipes, logs, and feedback; Alembic manages schema migrations.
Google Gemini (google-genai)	LLM generation — explanations, chef tips, recipe variations, and from-scratch recipe drafts.
sentence-transformers	Sentence embeddings powering semantic recipe search and nearest-neighbour ingredient substitution.

scikit-learn	The demand-forecast model and the learning-to-rank scoring model.
pandas	The data pipeline that cleans, parses, and enriches the source recipe dataset.
Next.js	The manager-facing analytics dashboard (summary, A/B results, catalog, feedback).
pytest	Extensive automated test suite spanning all five build phases.

Architecture

The backend is organised into clear layers: an **engine** (matching, scoring, ranking, embeddings, forecasting, feedback, weight learning), an **llm** layer (explanations, tips, creation, variations), a **context** layer (weather, events, trends, meal time), a **data_pipeline** (dataset cleaning and cost references), Pydantic **models**, and the FastAPI app that wires them together. Heavy data — the recipe catalog and ingredient lookup tables — is loaded once at startup, not per request.

The heart of the system is the recommendation pipeline: **FILTER** → **SCORE** → **RANK** → **LLM-EXPLAIN** → **LOG**. A request is first filtered to the dishes the kitchen can actually make, scored across many weighted dimensions, ranked, and only then — for the top few — passed to the LLM for a written explanation, keeping language-model calls (the slow, costly part) to a minimum.

```
# FILTER -> SCORE -> RANK -> LLM-EXPLAIN, per restaurant inventory + context
candidates = filter_recipes(_recipes, inventory_ids, ...)      # what can this kitchen make?
candidates = apply_venue_hard_filters(candidates, ...)        # dietary rules per venue

# Live context signals – each degrades to a neutral score on failure
weather = get_current_weather_tag(city)
events  = active_event_tags(date, ...)
trends  = get_trending_terms(city=city)

ranked = score_recipes(candidates, inventory, ...,             # 11 weighted dimensions
                      current_weather=weather,
                      forecast_scores=fc_scores,
                      weight_overrides=weights)

for entry in ranked[:max_results]:                            # LLM only for the top N
    explanation = generate_explanation(recipe, entry, ...)
    chef_tip    = generate_chef_tip(recipe, chef_profile)
```

Every request is tied to a restaurant by an `X-API-Key` header resolved against the database, so the platform is multi-tenant — each restaurant sees only its own data, and a separate admin key guards account-management endpoints.

The Scoring Engine

Each candidate dish is scored on eleven dimensions, combined into a single ranking score:

- **Margin** — cheaper-to-make dishes as a proxy for profitability.
- **Inventory relief** — how much current stock the dish uses up.

- **Expiry urgency (×2)** — favours dishes using soon-to-expire stock; weighted double, as waste is the most time-critical signal.
- **Weather / event / trend / meal-time fit** — context relevance from live signals.
- **Chef fit** — match to the chef's known cuisines and skill level.
- **Prep-time fit** — quicker dishes score higher.
- **Forecast fit** — predicted demand from the trained model.
- **Feedback boost** — a rolling 30-day signal from real outcomes (± 0.3).

The fixed-weight formula is only the baseline. Per restaurant, the system can apply **adaptive weights** learned from feedback, run an **A/B test** between weighting variants, or switch to a trained **learning-to-rank model** that predicts the probability a dish will be chosen — all behind the same endpoint, falling back gracefully when a model isn't trained.

Key Capabilities

Inventory intelligence

Matching is quantity-aware: recipe quantities and stock are normalised onto a common kg-equivalent scale, so the engine knows not just whether an ingredient is present but whether there's enough. When a dish is committed via `/confirm`, stock is deducted **FIFO by expiry** — soonest-to-expire batches are consumed first, directly supporting the waste-reduction goal.

```
# Deduct a recipe's needs, consuming soonest-to-expire batches first (FIFO)
sorted_inv = sorted(inventory, key=_expiry_sort_key)
for item in sorted_inv:
    need = remaining_kg.get(item.ingredient, 0.0)
    if need <= 0:
        updated.append(item); continue
    item_kg = item.qty * UNIT_TO_KG.get(item.unit, NO_UNIT_KG)
    if item_kg <= need:
        # batch fully consumed
        remaining_kg[item.ingredient] = need - item_kg
    else:
        # partial batch remains
        new_qty = item.qty - need / UNIT_TO_KG[item.unit]
        updated.append(item.model_copy(update={"qty": new_qty}))
```

Missing ingredients aren't always dealbreakers: using sentence embeddings, the engine finds a similar ingredient already in stock (a nearest-neighbour substitution) and, where stock is simply short of what a recipe needs, reports a "quantity gap" rather than dropping the dish.

AI generation

The LLM layer (Gemini) powers four features: a plain-language *explanation* for each recommendation, an optional *chef tip*, a `/variations` endpoint that suggests one-change tweaks using only in-stock items, and a `/create` endpoint that drafts an entirely new recipe from current inventory. Generated recipes are always marked as drafts for human review. Semantic `/search` lets a user find dishes by meaning ("something light and tangy for a rainy day") via embedding similarity rather than keywords.

The learning loop

After a dish is cooked, the kitchen logs the outcome through `/feedback`. Those outcomes feed three mechanisms: an immediate scoring boost for that recipe, the per-restaurant adaptive weights, and — once enough history accumulates — the demand-forecast and learning-to-rank models, both retrainable through admin endpoints. The system measurably gets better at each restaurant the more it is used.

Platform & data pipeline

Beyond recommendations, the project includes a full operational surface: admin endpoints to create restaurants and rotate API keys, a chef-driven catalog workflow (AI draft → submit → manager approve/reject, hot-reloaded into the live catalog without a restart), POS inventory sync, webhook and email notifications, and dashboard analytics endpoints consumed by the Next.js front-end. A separate data pipeline cleans and enriches a large Indian-food dataset into the recipe catalog and refreshes ingredient costs from live government market-price data.

Engineering Practices

The codebase is contract-first — a written API specification drives the Pydantic models, which in turn enforce request/response shapes and generate the OpenAPI documentation. Schema changes go through Alembic migrations. External signals (weather, trends, forecasts, the LLM) all **degrade gracefully**: if a service is unavailable, that signal falls back to a neutral value and the request still succeeds, with a warning attached. The whole system is backed by an extensive, phase-by-phase pytest suite covering matching, scoring, the AI features, multi-tenancy, and the learning loop.